
MLPC Report 2024

Team STITCH

Daniel Serdeshnyi

Roman Chervinskyy

Andrei Vasiliuk

Contributions

Roman - Data Preprocessing, Sliding window, Hyperparameter tuning

Andrew - Data Preprocessing, YOLO model.

Daniel - Naive Baseline system and managing Overleaf document.

1 Establish a naive Baseline system (i.e., a baseline without a classifier)

1.1 Describe what a naive baseline system for detecting speech commands in scenes could look like. How much cost can we expect from it?

Our first guess was the simple model that would identify words by scanning for volume above a specific threshold over a set period, i.e. simple decision tree, however, after implementing it we noticed that we could create a more naive and better classifier.

After careful examination of the cost function given and recreating possible scenarios of a classifier, we concluded that a simple classifier that never predicts anything would get a score of 595.0. It is due to the low cost of a False Negative in it. This simple model could outperform much more complex systems, therefore it should be considered a naive one to compare our models to.

Furthermore, we calculated that the minimal precision (with assumption that the recall is 1) is 0.63 for a model to outperform this Naive baseline. We suggest that poor RF or SVD implementation could not reach this threshold and would perform worse, which goes to show that such a naive Baseline is a great benchmark to compare our models to.

If a random classifier got correct timesteps of each command considering a task at a higher level that the difference between “an” and “aus” is absent, it would get a score of 3138. A model with slightly higher precision would still perform worse than Naive baseline. A model needs to reach a certain accuracy to outperform it.

2 A starting point: Build a simple speech command detection system (e.g., with one of the keyword classifiers you trained in the previous phase of the project).

2.1 Describe how your system was used to detect keywords in the longer domestic recordings (windowing, hop size, etc.)

We used a system proposed in our previous work. After simply applying our model to the dataset given we would get the results which are presented in Table 1. The architecture presented 1 works the following way: It goes through each time step and predicts a word, we can set step to be more than 1, we name it x , it has a running window of size s in which activation word “aus” or “an” must occur t times. If one of those words occurred, we look at the last k steps and look for the most frequent word from device words, if there is one and it occurred more than m times, then we say that activation word + activation word was said. x , s , t , k , m are all adjustable hyperparameters. They will be further discussed in section 3.

Table 1: Output of simply applying our model to the realistic scene "2_Florian_Heizung_aus" from Task 3

Frequency and time frames of occurring words			
Word	amount of frames	starting frame	starting second in recording
Alarm	3	111	2.78
Radio	6	483	12.08
Fernseher	2	489	12.23
Heizung	17	567	14.18
Leitung	1	584	14.60
Heizung	3	585	14.62
aus	17	613	15.33
Fernseher	1	770	19.25
wunderbar	1	772	19.30
warm	1	807	20.18
Alarm	1	808	20.20

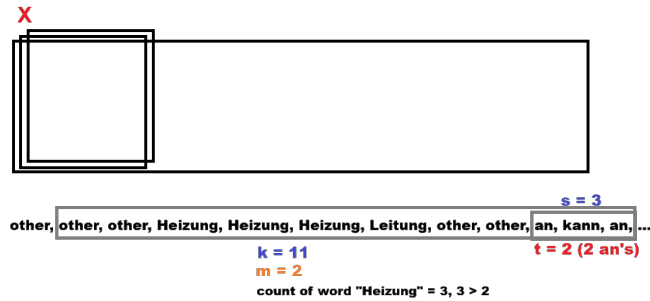


Figure 1: Architecture of our Sliding Window implementation

2.2 How did you threshold and combine keyword predictions to detect full speech commands? Describe any heuristics or post-processing strategies used.

We looked for the activation word ("an" or "aus") via a window of size s discussed in section 2.1. To be more precise an activation command must have occurred t times in the last s steps. That is also shown on the Figure 1. Then if there was a word detected more than m times in the last k steps that is labeled as a device word, we take this word. Then the function states: "device word + activation word" was said at the moment when a device word first occurs in the window of size k . Furthermore, we add to this timestamp 0.5 since since the cost function is adjusted so, that saying a word occurred earlier than later costs much more (our predictions are usually seeing the word much earlier than it actually occurred, cause we take into account white space before the word).

2.3 What strategies did you apply to minimize the task-specific cost function?

We adjusted our hyperparameters x , s , t , k , m so that FP would occur as little as possible. We aimed at the best Precision rather than Recall. We wrote a function that calculates precision and recall and on the development scenes we trained appropriate hyperparameters to maximize precision and still get a high Recall. We initially planned on introducing costs of specific FP, but our amount of total FP on development scenes was rather very low, hence those adjustments would not further improve our evaluations drastically. To be more precise, we had 9 actual FP on the development scenes evaluation.

2.4 Describe your evaluation setup and provide evaluation results on (parts of) the public, annotated scenes.

We calculated FP, TP, and FN. Wrongly classified words (i.e. say that one phrase was said instead of another) we calculated as both FN and FP. Our final results for the best model were: Recall: 0.5218, Precision: 0.9105, with TP: 621

Table 2: Results of Hyperparameter Tuning for Sliding Window Algorithm with $x = 2$, $k = 28$, m - min occurrences of a device work out of last k frames. (t, s) is amount of an activation word t needed to be said in a sliding window of size s .

m	metrics	(t,s)			
		(1,1)	(2,3)	(3,5)	(3,4)
1	Recall	0.5946	0.5702	0.5414	0.5399
	Precision	0.8221	0.8539	0.8798	0.8818
	Cost f.	-179.7	-203.1	-207.8	-209.3
2	Recall	0.5403	0.5218	0.4999	0.4983
	Precision	0.8868	0.9105	0.9246	0.9294
	Cost f.	-215.9	-221.7	-204.3	-202.4
3	Recall	0.4579	0.4471	0.4311	0.4294
	Precision	0.9252	0.9367	0.9501	0.9533
	Cost f.	-138.2	-133	-112.2	-110.3

out of 1190 Positives with a final cost of **-221.7**. Furthermore, as final results, we used the provided code for a more detailed outlook on our data. After that we wrote a function that returns some of the 10 random FN and 10 FP for a proper reasoning of the problems and further adjustments of the model.

2.5 Does your simple speech command detection system achieve lower costs compared to the naive Baseline system you established in 1.? If not, describe possible issues.

Yes, it achieved much better results. We believe that such a sliding window alone could achieve a good performance with the correct hyperparameters. The reason for that is the big amount of training data of specific words rather than real-life scenes with correct timesteps of both start and the end of a word.

To be more precise, the difference between our model and naive classifier is 816.7 in terms of a given cost function.

3 Improvements: Investigate at least three diverse strategies to improve your starting point (e.g., via hyperparameter tuning, ensembling, data augmentation, cost-specific tuning, post-processing, using data from the new development set for training, etc.)

3.1 For each of these strategies: Describe the main ideas you had and hypotheses you tried, as well as their outcome after experimentally verifying or falsifying the hypotheses (via validation on (parts of) the development set). Do not hesitate to report negative outcomes.

1. **Hyperparameter tuning of the sliding window discussed above with respect to high Precision:** Due to the specifics of the data, we believed that the sliding window architecture described above would work on this task nicely due to a relatively big amount of training samples. We decided to investigate which hyperparameters of our sliding window would work the best way. For k last steps we decided to take the upper quartile of the difference between the start of "aus/an" and the device word. It was 50, hence we used $50/x$ frames, in the case with $x = 2$, we use $k = 25$. Further investigation concluded that to not miss some of the outliers k should be 28. During our training, we assumed that frame step of 2 was the best tradeoff between complexity and speed of the algorithm. Furthermore, we exclude some values that would obviously result in a high cost. After hyperparameter tuning, we got Table 2 and picked the model with the lowest cost on training data.
2. **YOLO:** Since our dataset is a collection of single words with all the respective features and the final goal is to identify a command, consisting of multiple words, we decided to try out the detection approach. The idea is rather simple: we train our model to detect the words in a spectrogram and after getting their location - perform a classification task on a cropped bounding box of a word. For this task we chose **YOLOv5** - the model that can do both of the tasks required: identification of the location of a word and its classification. The problem was in compounding the dataset and getting bbox labels for the words we already have. We can't do it straightforwardly, because our spectrogram includes not solely the word, but also some "empty" frames before and after the word. To handle it we calculated the mean of each timestamp and after selecting gradient peaks we cropped our word. The whole procedure is shown on the Figure 2.
3. **Specific Data Preprocessing:** Furthermore, after labeling all the words we also introduced random noise samples before and after the words as well as mixed them to compound the phrases. We did it speaker-wise:

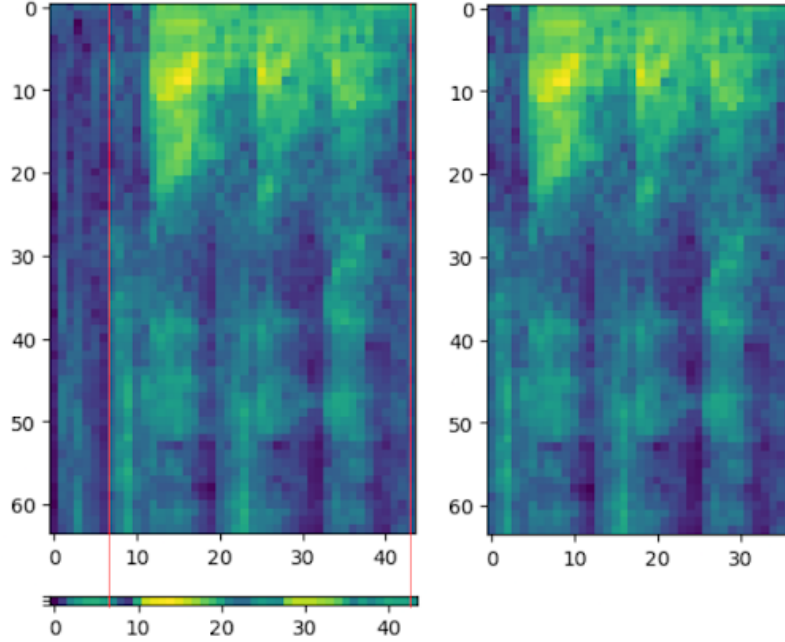


Figure 2: Procedure of cropping the word after selecting gradient peaks

we took random noise of arbitrary length from 1 to 44 and placed it in between the words said by this user. In theory, it could give us $\binom{n}{k}$ (n choose k) samples for every user, but because of the limitations of our CPU, we decided to do it for every word separately. Since the median count of words “aus” and “an” is lower than the median count of device words (24 to 96 respectively), we reused those activation several times. After all data-preprocessing, our data looked like Figure 3.

It is not a perfect data representation of the samples that will be given for testing, but since we were not given the specific data required for YOLO, we labeled the created dataset as described above and trained our model using different hyperparameters. The model achieved Precision = 0.92 and Recall = 0.89 on the test dataset after the train-validation split. When we tried it on the new data given for testing, the scoring was -15. This approach is more stable and scalable than the sliding window and can be further improved by introducing autoregression learning and data augmentations, but due to the lack of time, we didn’t manage to integrate it with the sliding window and make any improvements. As mentioned above, the problem is that it is not working as effectively as on the initial dataset because the difference between our training data and the unseen data we want to predict is high.

4 Critical Reflection

4.1 Do you think your final system could be deployed in a real-world application? In your opinion, which aspects of the project or your system would need to be adapted to fulfill possible real-world requirements?

Yes, we believe that our system 3 could indeed be implemented in real-life scenarios for that task. We believe that with more real-life-like data we could further improve our model and it would work without big flaws. We would need to adjust it to the continuous flow of data and further adjustments. Our current problem is the lack of real-life-like scenarios for training and a big difference between the data initially given and the final goal. Our model might be too complex for a smart-home system, therefore we also would need to make it more simple, as now for a sliding window with a batch size of 44 it takes 14.16 minutes to go through all development scenes for validation on the GTX 1660 Ti. As for YOLO it takes: 1.3 minutes for the same dataset on RTX 2060, and for 2 systems combined (YOLO-SW): 12.5 minutes respectively.

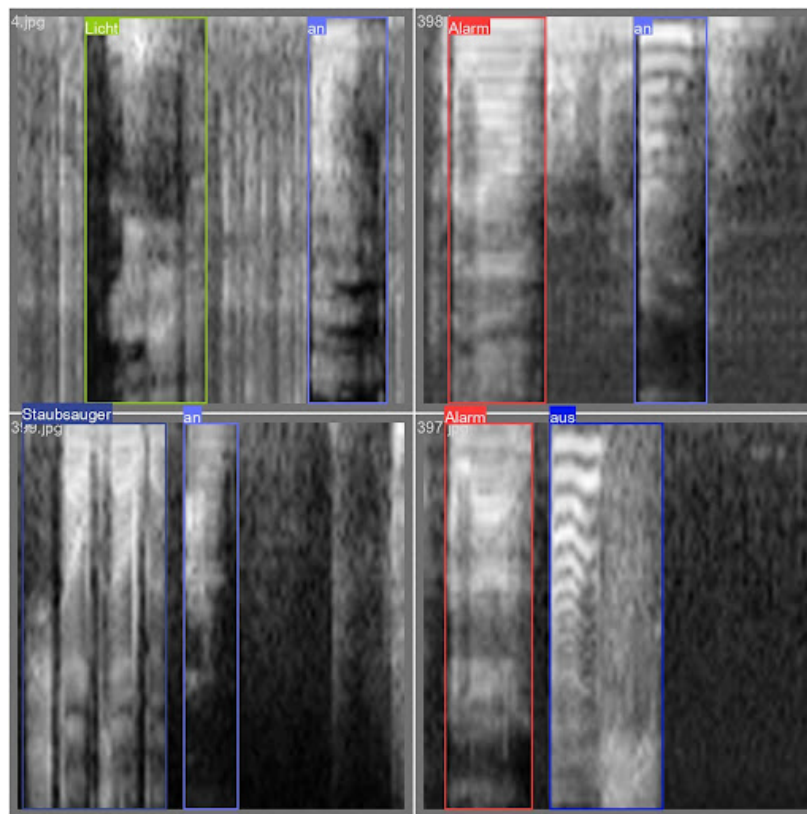


Figure 3: Illustration of data after pre-processing